

Data Mining Final Project: Finding Similar Netflix Users with Locality Sensitive Hashing

S. Nehzati
S. Moghiseh

T. Ghaneh
P. Popp

November 2025

1 Introduction

This project focuses on finding pairs of similar users in a large Netflix dataset. We want to identify users who have rated many of the same movies, using Jaccard similarity as our measure. Jaccard similarity compares two sets by dividing the number of items they share by the total number of different items in both sets combined. In our case, we look at which movies each user has rated, ignoring the actual rating values.

The dataset contains over one hundred thousand users who together rated almost eighteen thousand movies, creating more than sixty-five million rating records. If we tried to compare every possible pair of users directly, we would need to perform billions of comparisons. This brute force approach is not practical because it would take too long and use too much computer memory. Instead, we use algorithms based on hashing and grouping that can find similar pairs much more efficiently.

Specifically, we use two complementary techniques. First, we compress each user's movie list into a much shorter signature using a method called Minhash. Second, we use Locality Sensitive Hashing to group users with similar signatures together, so we only need to check pairs within the same group. These techniques together make the problem manageable. We implemented these algorithms in Python, aiming to find all user pairs with Jaccard similarity greater than 0.5 while keeping the program runtime under thirty minutes on a standard computer. Our implementation successfully finds between one hundred fifty and two hundred similar user pairs in about 10 to 15 minutes, demonstrating the effectiveness of these techniques.

2 Problem Statement

Our task is to find all pairs of users whose similarity exceeds a threshold of 0.5. Two users are similar if they rated many of the same movies. We ignore what rating they gave and only care about which movies appear in their lists. This means if two users both rated fifty movies and forty of those movies are the same, their Jaccard similarity would be quite high, even if the ratings they gave are different. Another consequence of the data is that there are not many user-movie overlaps, meaning that we will have to deal with sparsity. This raises the problem of finding an optimal input data representation, that converts the ratings in the right format and stores the data in a memory-efficient way.

Another task is to find the optimal parameters for the two algorithms used. The outcome of the Minhash algorithm depends on the number of signatures, whereas the success of the LSH depends on carefully chosen number of bands and rows, and the number and size of the buckets. This includes balancing the precision and recall associated with the parameters, and we need to take into account that our goal is to find as many 50% similar pairs as possible.

Finally, it is crucial to determine a time-efficient procedure for the post-processing of the candidate pairs found by LSH. The procedure should be able to handle a big amount of comparisons and only write the pairs that are in fact more than 50% similar.

3 Methodology

3.1 Input data representation

The original data is provided as an array of integers, where each row contains the ID of the user, the ID of the movie, and the given rating. Multiple rows correspond to the same user, and the same movies appear in multiple rows as well. Before proceeding to the algorithm, the data has to be converted to a suitable format, which in this case is a matrix where the rows represent movies and the columns represent users. Importantly, we expect this matrix to be sparse (have many zero entries), as many users rated only a fraction of all the movies in the data and the matrix will thus be largely empty. In this scenario simply converting the input data to such a matrix would be unreasonable, as the memory will be wasted on storing the many zero entries.

We therefore chose to represent the input data as a sparse matrix that only stores non-zero elements: it will only store the positions where a user has rated a movie, ignoring all the empty spaces. This lets us handle the large dataset without running out of memory. The conversion was done by using the *sparse* package of the *SciPy* library. Specifically, the Compressed Sparse Row format (CSR) was chosen. It provides efficient row-wise operations, which are necessary for the random permutations we apply during Minhash computation.

When building the sparse matrix, we paid attention to the data types. The arrays storing the user and movie IDs, used in forming the matrix, were cast as 32-bit integers to save memory. The rating values were cast to boolean data type, since we only care about whether a movie was rated or not. The data type of the CSR matrix was set to boolean as well, which is the most memory efficient choice.

3.2 Minhashing

While the CSR data representation is memory efficient, it is still too large for us to compute similarities efficiently. The Minhash algorithm offers a solution: it works by creating a short signature for each user, that preserves the similarities between them. The signature is built as following. First, the rows (the movies in our case) are rearranged in random order. Next, for each rearrangement, we find the first movie that the user rated and record its position. After doing this many times with different random arrangements, we have a vector of numbers that represents the user. The key property of Minhash is that if two users have high Jaccard similarity, their signature vectors will also be very similar. This lets us estimate similarity by comparing short signatures instead of long movie lists.

Our implementation uses 80 random permutations to build signatures of length 80 for each user. The reasoning behind the choice of this number as well as the other algorithm parameters is presented in Section 4. The signature matrix M was preallocated as an array with 80 rows (chosen number of permutations) and the number of columns corresponding to the number of users. The data type was set to 32-bit integer for memory efficiency.

For each permutation, we generate a random ordering of all movie indices using the permutation function offered in the *NumPy* library. We then rearrange the sparse matrix according to this ordering and use the *argmax* function to find the first occurrence of a rated movie for each user. This vectorized approach processes all users at once, which is much faster than looping through users individually. This sequence of the first occurrences is then written as a row of the signature matrix M , corresponding to the result of one permutation.

3.3 Locality Sensitive Hashing

After creating signatures for all users, we apply Locality Sensitive Hashing. This technique divides the signature matrix into several bands, where each band contains a few consecutive rows from the signature matrix. In each band, the users are assigned into buckets, based on the values they have in the matrix M in the chosen band. If two users have identical values in a band, they are placed in the same bucket and become candidates for similarity checking. The idea is that truly similar users will likely match in at least one band, while dissimilar users probably will not match in any band.

The number of bands (b), the size of each band (r) and the maximum bucket size (number of collisions allowed per bucket) are critical parameters that determine the algorithm's behavior. The values of b and r

(20 and 4 respectively) were found in an optimization procedure, and the maximum bucket size (100) was determined by running a number of experiments. These procedures are elaborated on in Section 4.

In our implementation, first, we created an empty set to store the candidate pairs. The set type helps to ensure that no duplicate pairs are stored. For each band, the corresponding rows are chosen, and a bucket dictionary is created. The number of keys in the bucket dictionary is set to the number of users, which is done for two reasons. First, using less buckets would raise the risk of artificially creating more collisions. Second, using more buckets did not lead to a big change in neither the number of buckets that have many collisions (more than 100 users) or the number of buckets with 2 users only.

Further, we used a simple hash function to convert each user’s band rows into a bucket identifier. For each user in each band, we extract the corresponding portion of their signature, convert it to a tuple of integers, hash it using Python’s built-in *hash* function, and take the result modulo the number of buckets. This operation is applied to all users in a vectorized way using the *apply along axis* function from *NumPy*, to speed up the computation. As a result, each user in the band is assigned a bucket ID. Finally, we fill the bucket dictionary using the bucket ID as key, and the users as values. If there are at least 2 and less than 100 users in a bucket, the pair (or all possible pairs, in case of more than 2 users) is sorted and appended to the set of candidate pairs.

This procedure is repeated for all bands. With the current settings, we consistently obtain more than 11 million candidate pairs.

3.4 Post-processing

Once we have the candidate pairs from the banding process, we need to verify whether these pairs are actually similar. An important step that significantly reduced the computation time at this stage is converting the original CSR matrix (movies x users) to its transpose (users x movies). This allows us to compute the Jaccard similarity quicker, since we can index the matrix row-wise and reap the advantages of the CSR format. The original CSR matrix was overwritten to save memory, since we will not be using it afterwards.

We implemented a two-stage filtering process to find the real similar pairs. First, we compute the signature similarity by comparing how many positions in the eighty-element signatures have matching values. If this signature similarity does not exceed 0.5, we skip the pair immediately without loading any data from the sparse matrix. Only if the signatures are similar enough do we proceed to the second stage.

In the second stage, we load the actual movie sets for both users from the sparse matrix (CSR format) and compute the exact Jaccard similarity. We count how many movies both users rated using set intersection, count the total number of different movies using set union, and divide the intersection by the union to get the Jaccard similarity. Only pairs where this exact similarity exceeds 0.5 are written to the output file.

This process is completed rather quickly, and therefore all buckets were tested. This was done to minimize the risk of missing candidate pairs, as our aim is to find as many 50% similar pairs as possible.

The effectiveness of the two-stage filtering is demonstrated by the numbers at each stage. LSH typically generates millions of candidate pairs from all the buckets combined. The first filter, based on signature similarity, eliminates the largest part of these candidates very quickly. Only a few thousand pairs pass this filter and require loading data from the sparse matrix. Of those, the exact Jaccard computation finds that between 150 and 200 pairs (depending on the random state) actually exceed the 0.5 threshold. For instance, with the seed of 42, the program found 151 pairs. This shows that the filtering is working well to focus computational effort on the most promising candidates.

4 Parameter Selection

A parameter that influences the outcome of the Minhash algorithm is the **signature length**, or the number of permutations. The value of 80 was chosen to balance accuracy and speed. Fewer permutations would be faster but less accurate in estimating similarity, whereas more permutations would be more accurate but take longer to compute. We tested values from 60 to 100 and found that 80 provided a good compromise. The computation of the signature matrix using 80 permutations takes about 6 to 7 minutes on a standard laptop, which is acceptable.

Further, the **number of bands and rows** for the LSH algorithm was chosen. The key consideration in this choice is our aim to find as many pairs as possible. This could be translated to trading in precision

in favor of recall. For instance, we would rather generate many pairs (including some false positives) and keep the false negative rate low. The false positives would then be filtered out in post-processing. With the current aim, this would be more favorable than generating fewer pairs with a lower amount of false positives, but a higher risk of missing the similar pairs.

To determine the parameter values, we could use the formula $1 - (1 - s^r)^b$, where s is the similarity threshold, and b and r are the numbers of bands and rows. This formula denotes the share of the pairs we would find using the chosen b and r , if the pairs are 50% similar. Taking our reasoning into account, we would want this number to be high, so that we would find as many pairs with the similarity of 0.5 as possible. We could therefore view the parameter choice as an optimization problem, where the formula above is the objective we aim to maximize. However, we first need to introduce some constraints. First, the product of bands times rows cannot exceed the signature length of 80. Second, the product must cover at least 90% of the signature to avoid wasting information. Third, importantly, we must set a limitation on the number of rows. Using 1 row will give us the highest "detection" rate, but also yield a very high number of false positives and come with an unreasonable computational cost. We therefore define that the number of rows per band must be greater than 3.

For each valid combination of b and r , we calculate the probability of detecting a pair that is 50% similar using the formula presented above. We select the combination that maximizes this detection probability. With our signature length of 80, the optimization selects 20 bands with 4 rows each, giving a detection probability of about 72% and a false negative rate (computed as $(1 - s^r)^b$) of around 28%. The false negative rate seems high, yet, we found that this configuration produces quite a lot of false positives, and reducing the false negative rate further by taking a smaller r would likely yield mostly an unreasonable increase of computations.

To further investigate whether these values represent a reasonable tradeoff between precision and recall, we also examined the threshold at which the probability of detection would rise at the given similarity level. The formula denoting this threshold is $(\frac{1}{b})^{\frac{1}{r}}$, where b and r are the corresponding number of bands and rows. We set the same constraints as above, besides relaxing the constraint on r (greater than 2, instead of greater than 3) to allow for more exploration. We ran a quick loop through all valid values of b and r and calculated the threshold. The values that yield the smallest difference between the computed threshold and 0.5 (our desired similarity) are recorded as the best choice. This optimization yielded a threshold of 0.48, associated with 19 bands and 4 rows, which is very close to the values found above. Yet, the detection probability achieved with these values is slightly lower (71%), so that we chose to proceed with the values of 20 bands and 4 rows.

This approach helped to make the code more robust and easier to use with different settings, such as the number of permutations. The optimization (in both cases) runs very quickly, taking only a fraction of a second, because it only needs to test a small number of valid combinations. Only the first optimization (of the detection probability directly) was left in the final code, since it yields the values that are subsequently used.

The **maximum bucket size** of 100 was chosen based on computational cost considerations. A bucket of size k would require $k(k - 1)/2$ comparisons, and if k is set to 100, we would thus have to compare almost 5000 pairs. Larger buckets would create many more - tens of thousands - pairs from a single bucket, which would take too long to compare. Smaller limits would be faster but bear a higher risk of missing some pairs. We found that the bucket size of 100 provided good results without excessive runtime. Buckets larger than this limit are skipped entirely during candidate generation. This trades off a small number of potential matches for a large gain in speed and keeps the program runtime manageable.

5 Implementation Details

Our implementation is structured as a series of well-documented functions, each handling one major step of the algorithm. The program begins by loading the data from a NumPy file and building the sparse matrix representation. It then computes the Minhash signature matrix, automatically determines the optimal banding parameters, performs LSH to generate candidate pairs, and finally verifies candidates to find true matches.

In the implementation, we also had to ensure reproducibility, since the minhashing algorithm includes

random permutations. The random seed is a critical parameter for reproducibility. At the start of execution, the program prompts the user to enter a random seed value. This seed is used to initialize NumPy’s random number generator, which controls all the random permutations. Running the program multiple times with the same seed produces identical results, meaning that the same pairs of similar users will be found. Different seeds produce different permutations and therefore different signatures, leading to some variation in which pairs are found.

Memory management was carefully considered throughout the implementation. We use 32-bit integers for indices instead of the default 64-bit integers, which cuts memory usage in half for these arrays. The sparse matrix uses boolean type for the ratings, which is the most compact representation. After building the sparse matrix, we no longer need the original data array, and therefore delete the variable containing the data to save memory.

6 Results and Discussion

We tested our implementation on the provided dataset with several different random seeds to verify robustness and understand variation. The program consistently completed in under 15 minutes on standard laptop hardware with 8 gigabytes of RAM. Across different random seeds, we found between 150 and 200 pairs of users with Jaccard similarity greater than 0.5. This variation is expected and acceptable, as different random permutations naturally lead to different signature values and different candidate sets. The number of found pairs is on the higher side across all random states, which is desirable since we aimed to find as many pairs as possible. The pairs that were found at the random state of 42 and their Jaccard similarities (computed on the original dataset in the CSR format) are presented in Table 1 and Table 2.

The runtime breaks down into several distinct phases with different computational costs. Loading the data and building the sparse matrix takes about 20 to 30 seconds. This involves reading the large NumPy file, extracting user and movie IDs, and constructing the sparse representation. Computing the Minhash signatures takes the most time, typically around 6 minutes. This phase must process all seventeen thousand movies and perform eighty permutations, each involving a full matrix rearrangement. The optimization of the bands and rows parameters is very fast, taking less than a second. The LSH banding and bucket creation tends to complete in under a minute, because it involves simple array slicing and hashing operations. Finally, checking candidate pairs takes between 2 and 6 minutes depending on how many candidates are generated and how many pass the signature similarity filter.

Several conclusions can be made based on the experiments performed. First, we witnessed how the conversion to CSR significantly sped up row-wise operations, both in the permutation stage during Minhashing and in the computation of Jaccard similarity in post-processing. Second, in the context of the relatively time-consuming Minhashing algorithm, using a lower number of permutations showed a good performance and allowed to reduce the number of computations. Third, the LSH algorithm tuned for higher recall (as we aimed to find as many pairs as possible) produced a very big number of candidate pairs, and most of them were filtered out in post-processing. However, this did not prove to be a disadvantage in the end, as the two-stage filtering in post-processing appeared to be a very quick procedure.

ID of User1	ID of User2	Jaccard similarity
47930	78973	0.52
66759	94019	0.51
38031	80001	0.51
60285	92817	0.5
1372	19440	0.5
9018	75835	0.54
18867	56569	0.53
20444	99433	0.51
7624	10583	0.53
7873	91352	0.5
13621	49651	0.59
7624	54549	0.51
7624	15039	0.51
63990	73960	0.59
33367	75289	0.51
92817	102069	0.5
10233	47113	0.5
47113	100549	0.53
19440	66240	0.5
32934	46528	0.51
28634	50547	0.51
74191	88887	0.5
10612	53267	0.5
53267	70560	0.51
18693	40860	0.51
70292	78517	0.5
49459	68620	0.5
50547	101404	0.51
64421	78517	0.51
51237	82603	0.5
80239	81963	0.53
7624	70292	0.51
15039	20444	0.5
53267	62898	0.51
10583	18867	0.52
52774	58328	0.51
414	82715	0.53
47660	47930	0.5
18867	94019	0.54
18867	82603	0.5
46205	99433	0.5
53267	64421	0.53
52432	53267	0.52
8002	70718	0.51
44623	46365	0.5
16194	74191	0.53
33367	64421	0.51
19153	68324	0.51
7873	102559	0.5
52432	101404	0.51
43100	91352	0.5
78517	91158	0.54
14779	69562	0.51
7624	94607	0.51
58328	77807	0.51
47930	73569	0.53
50459	60183	0.59
69680	74191	0.51
9772	51237	0.51
34006	81963	0.52
1789	7624	0.52
9772	21638	0.5
9549	81963	0.52
18481	81963	0.5
38031	99433	0.51
78517	96625	0.5
10583	16194	0.52
51237	78517	0.52
19153	83313	0.51
55064	73538	0.52
58328	91947	0.51
20444	74191	0.53
34006	94019	0.5
51237	83313	0.53
8400	18867	0.51

Table 1: User similarities (first 75 pairs)

ID of User1	ID of User2	Jaccard similarity
36825	91532	0.51
34828	53267	0.52
378	103532	0.51
15039	33146	0.53
58328	68324	0.5
7624	15960	0.5
47930	90660	0.56
414	18481	0.52
8002	46365	0.5
20789	61541	0.55
33146	51237	0.5
30525	55033	0.72
43129	56268	0.54
38299	88887	0.51
35419	53267	0.54
52774	53267	0.53
10583	53267	0.53
7624	61541	0.51
414	83057	0.5
81963	100951	0.52
37278	96625	0.51
10583	78517	0.52
59757	76985	0.54
16194	29807	0.53
19153	94428	0.5
33367	52774	0.51
74191	97494	0.51
46365	74191	0.52
7624	48525	0.51
15039	74191	0.54
68875	91352	0.51
16014	51237	0.5
15039	61877	0.52
74191	78269	0.5
74875	79321	0.53
53267	78517	0.56
47930	91352	0.53
15039	18867	0.51
32672	100079	0.51
9018	18382	0.51
73538	89493	0.5
414	16194	0.55
33367	53267	0.54
98295	98624	0.5
42843	97494	0.51
7873	71374	0.51
47930	101404	0.54
18388	98688	0.51
35419	85467	0.52
33367	47113	0.5
52495	99904	0.84
66759	81963	0.52
43039	94019	0.5
71303	75160	0.53
15162	47930	0.54
28512	58328	0.51
78269	101404	0.5
42122	43039	0.5
10583	33367	0.53
18867	62898	0.55
91352	102559	0.52
36825	77807	0.5
15039	16194	0.54
9772	78517	0.53
33146	50436	0.52
81963	90297	0.53
20164	85467	0.51
91352	99433	0.51
19153	58328	0.5
7624	74191	0.54
16194	55105	0.52
64421	70292	0.51
85093	87011	0.51
24950	66327	0.51
53267	90297	0.5
8296	30251	0.52

Table 2: User similarities (last 76 pairs)